Certified Block Screening for Adaptive Long-Context Attention

Marcel
Independent Researcher

April 19, 2026

Abstract

Long-context dense attention is expensive at inference time, and aggressive approximate-selection methods often fail unpredictably when the exact-attention budget is tight. We propose a runtime adaptive-attention mechanism that partitions the KV cache into fixed-size blocks of length b, applies a coarse global path, and selects a small subset of $k \ll m = \lceil n/b \rceil$ blocks for exact attention using a certified upper bound on each block's best possible query–key score. The bound is derived from a one-line Cauchy–Schwarz argument on per-block center and radius metadata, $U_r(q) = q^\top c_r + \|q\|_2 \rho_r$, and we observe zero violations across every configuration probed. On Qwen3-0.6B [Qwen Team, 2025], bound-screened adaptive attention achieves 1.07×, 1.38×, 2.24×, and 5.30× prefill speedups over a dense SDPA [Dao et al., 2022] baseline at 4k, 8k, 16k, and 32k context respectively, with no measurable calibration-perplexity loss. Under tight exact budgets, certified screening degrades substantially more gracefully than naive fixed top-k selection: at $k = 5$, fixed top-k collapses to a calibration perplexity of 5,689, while bound screening holds at 33. We also report a negative result: coarse block screening at 64-token granularity fails single-token passkey retrieval [Liu et al., 2024], and a multi-prototype block-summary extension does not rescue retrieval at the coarse selection resolution we use. These findings position certified block screening as a practical runtime method for long-context efficiency, not a general retrieval architecture.

Framing sentence. This is a certified runtime screening method for long-context efficiency, not a general retrieval architecture.

1 Introduction

Long-context inference with dense attention scales quadratically with the KV length n: each new query performs $O(n \cdot d)$ key–value operations against the entire cache, for a prefill cost of $O(n^2 d)$. Runtime-efficient alternatives trade exactness for structure: sliding-window locality [Beltagy et al., 2020, Zaheer et al., 2020], compression-based attention [Wang et al., 2020], IO-aware exact kernels [Dao et al., 2022], or heuristic query-aware selection [Tang et al., 2024, Jiang et al., 2024, Fu et al., 2024]. Each compromise is acceptable in its intended regime, but methods that rank blocks of keys by an unbacked empirical summary score can degrade catastrophically when the summary is uninformative for a given query — a failure mode we observe directly in §5.2.

There is room for a method that (i) keeps the exact KV cache available, (ii) spends cheap global compute on a coarse summary pass, and (iii) screens which blocks deserve token-level exact attention using a *certificate* of possible relevance, not merely a heuristic estimate of it. We want the certificate to be provably sound, inexpensive to compute and maintain across decoding steps, and selective enough to produce a measurable shift in selection when compared against a heuristic baseline.

We propose *certified block screening.* Each KV block is summarised by a center $c_r \in \mathbb{R}^d$ and a scalar radius $\rho_r \in \mathbb{R}_{\geq 0}$ satisfying $\|k_j - c_r\|_2 \leq \rho_r$ for every key in the block (§3.3); a query q is

screened against blocks via the Cauchy–Schwarz upper bound

$$U_r(q) \;=\; q^\top c_r \;+\; \|q\|_2 \rho_r,$$

which provably upper-bounds the best token-level score $\max_{j \in B_r} q^\top k_j$ inside the block. We implement the selector as a drop-in replacement inside an existing dense model (Qwen3-0.6B [Qwen Team, 2025]) with a shared-across-queries gather path that lets the post-screening attention run as a single fused kernel call (§3.5). On this model, certified screening delivers prefill speedups that grow with context length, reaching 5.30× at 32k tokens (§5.1); calibration perplexity is preserved within run-to-run jitter; and the selector's failure mode under tight exact budgets is qualitatively softer than that of heuristic top-k (§5.2). We also identify a clear limitation: coarse block selection at 64-token granularity is not sufficient for sparse single-token retrieval (§6), and we document a negative multi-prototype experiment that demonstrates which of summary resolution and selection resolution is the binding constraint.

Contributions.

- A certified runtime block-screening selector for adaptive long-context attention, with a one-line provable upper bound $U_r(q) = q^\top c_r + \|q\|_2 \rho_r$ and cheap per-block metadata (one center plus one scalar radius).
- An empirical demonstration that certified screening improves graceful degradation under tight exact budgets and yields a growing latency–quality advantage over dense attention as context scales to 32k tokens, while preserving calibration perplexity.
- A careful negative-result analysis showing that coarse block screening at the tested granularity is insufficient for sparse single-token retrieval, and that simple multi-prototype extensions do not rescue retrieval because selection granularity, not summary granularity, is the limiting axis.

2 Related Work

Efficient long-context attention. FlashAttention [Dao et al., 2022] and its ROCm/AOTriton descendants make dense attention IO-efficient but leave the $O(n^2)$ arithmetic intact. Linear-attention and compression methods [Wang et al., 2020] bound memory and throughput by restricting the attention operator; they operate in a different approximation regime than token-level exact attention. Sliding-window and block-sparse patterns [Beltagy et al., 2020, Zaheer et al., 2020] hard-code a locality prior, which is orthogonal to the runtime-selective problem we address.

Query-aware selective KV retrieval. The most directly related prior work screens or routes KV information at inference time using query-dependent signals. Quest [Tang et al., 2024] maintains per-page element-wise minimum and maximum key values and scores each page against those bounds. MInference [Jiang et al., 2024] routes queries to structured sparsity patterns selected offline. LazyLLM [Fu et al., 2024] defers and prunes KV cache tokens using importance heuristics derived from past attention scores. SeerAttention [Gao et al., 2024] learns intrinsic block-level sparsity inside the attention map. DuoAttention [Xiao et al., 2024] explicitly separates retrieval heads (which keep the full KV cache) from streaming heads (which do not), a head-level complement to our block-level selection. All of these systems perform selection during inference on top of an unmodified pretrained model; our method shares that setting but differs in the selector's metadata and soundness guarantee, discussed in §3.3.

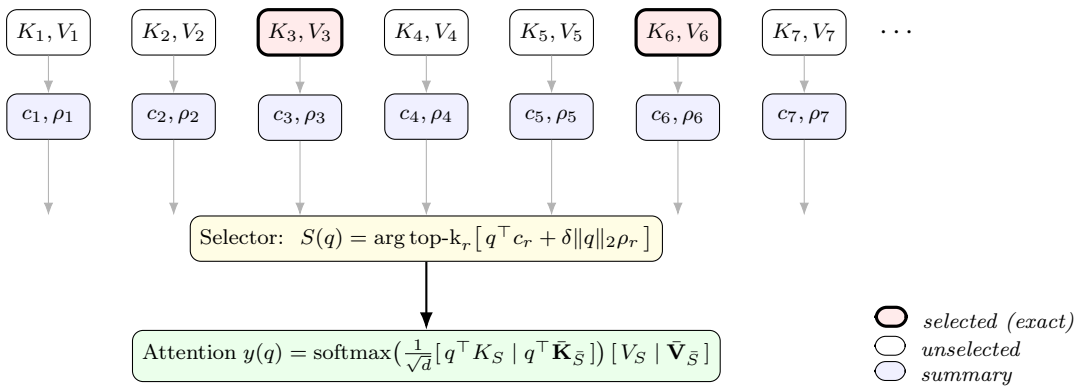

Figure 1: Method overview (Fig. 1). The KV cache is partitioned into fixed-size blocks. Each block carries a certified summary (c_r, ρ_r) with $\|k_j - c_r\|_2 \le \rho_r$ for every key $k_j \in B_r$. A query scores all blocks via $U_r(q) = q^\top c_r + \delta \|q\|_2 \rho_r$ (Prop. 1), selects a small top-k subset for exact token-level attention, and feeds the rest as summaries in a single joint softmax.

What is distinct here. The closest prior method, Quest [Tang et al., 2024], ranks pages by element-wise min/max metadata, which yields a tight per-token upper bound but requires $2d$ scalars per block and a component-wise scoring op. Our selector stores one center and one scalar radius per block — total $d{+}1$ scalars — and derives its bound from Cauchy–Schwarz in one line. Neither design point strictly dominates the other: Quest's metadata are finer-grained at the cost of more scoring work and more memory, while ours are coarser and cheaper but still certified. We contribute three things in addition to a different selector: (i) a per-block selector whose decisions are backed by a provable upper bound; (ii) an explicit measurement of that bound's violation rate on held-out data (zero across every configuration we probe, §5.3); and (iii) a characterisation of the under-budget regime, which prior efficient-attention work has tended to leave implicit. We do not claim category-level novelty relative to query-aware sparse attention broadly, and we compare against the natural FixedTopK selector directly throughout §5.

3 Method

3.1 Setup

Let $n \in \mathbb{N}$ denote the KV-cache length at the point of attention and $d \in \mathbb{N}$ the per-head hidden dimension. We partition the sequence axis into $m = \lceil n/b \rceil$ fixed-size blocks of size b, padding the final block by repeating its last entry. Formally, block $r \in \{1, \ldots, m\}$ carries the token index set

$$B_r \;=\; \big\{(r-1)b,\, (r-1)b+1,\, \ldots,\, \min(rb-1,\, n-1)\big\}$$

and the corresponding key and value matrices $K_r \in \mathbb{R}^{b \times d}$, $V_r \in \mathbb{R}^{b \times d}$. Let $k \in \mathbb{N}$, $k \ll m$, denote the *exact-attention budget* — the maximum number of blocks whose per-token keys and values will participate in the token-level softmax. Grouped-query attention is handled by the standard repeat of KV heads across query heads; the method is described per query head.

3.2 Coarse-to-exact attention

For each block r we maintain a summary $(\bar{k}_r, \bar{v}_r) \in \mathbb{R}^d \times \mathbb{R}^d$, taken as the per-head mean

$$\bar{k}_r \;=\; \frac{1}{|B_r|} \sum_{j \in B_r} k_j, \qquad \bar{v}_r \;=\; \frac{1}{|B_r|} \sum_{j \in B_r} v_j. \tag{1}$$

Given a query $q \in \mathbb{R}^d$ and a selection $S(q) \subseteq \{1, \ldots, m\}$ with $|S(q)| \leq k$, the attention output is

$$y(q) \;=\; \mathrm{softmax}\Big(\tfrac{1}{\sqrt{d}}\,\big[\,q^\top K_{S(q)} \;\big|\; q^\top \bar{\mathbf{K}}_{\bar{S}(q)}\,\big]\Big) \cdot \big[\,V_{S(q)} \;\big|\; \bar{\mathbf{V}}_{\bar{S}(q)}\,\big], \tag{2}$$

where $K_{S(q)} \in \mathbb{R}^{|S|b \times d}$ is the row-concatenation of exact token keys from the selected blocks, $V_{S(q)}$ the corresponding values, and $\bar{\mathbf{K}}_{\bar{S}(q)} \in \mathbb{R}^{|\bar{S}| \times d}$, $\bar{\mathbf{V}}_{\bar{S}(q)}$ collect the summaries of the unselected blocks $\bar{S}(q) = \{1, \ldots, m\} \setminus S(q)$. We refer to the two halves of this joint softmax as the *exact-refine* and *coarse* paths:

$$y(q) \;=\; \underbrace{y_{\text{coarse}}(q)}_{\text{over } \{(\bar{k}_r, \bar{v}_r) : r \in \bar{S}\}} \;+\; \underbrace{y_{\text{exact-refine}}(q)}_{\text{over } \{(k_j, v_j) : r \in S,\, j \in B_r\}} \qquad \text{(up to the shared softmax normaliser).} \tag{3}$$

In practice we evaluate the joint softmax with a single fused kernel call; the decomposition (3) is conceptual. Two invariants matter: the exact KV cache is never discarded, so only the *allocation* of exact attention varies per query; and the softmax is causal and well-defined for every query, including those whose selected blocks lie entirely in the future.

3.3 Certified block screening

Definition 1 (block summary with radius)**.** A *certified block summary* for block r is a pair $(c_r, \rho_r) \in \mathbb{R}^d \times \mathbb{R}_{\geq 0}$ satisfying

$$\|k_j - c_r\|_2 \;\leq\; \rho_r \quad \forall\, j \in B_r. \tag{4}$$

We take $c_r = \bar{k}_r$ from (1) and $\rho_r = \max_{j \in B_r} \|k_j - c_r\|_2$. Both are computed in bf16 with fp32 accumulation of the squared distances:

$$\rho_r^2 \;=\; \max_{j \in B_r} \sum_{\ell=1}^{d} (k_{j,\ell} - c_{r,\ell})^2. \tag{5}$$

Proposition 1 (certified upper bound)**.** *For every query $q \in \mathbb{R}^d$ and every key $k_j \in B_r$,*

$$q^\top k_j \;\leq\; q^\top c_r \;+\; \|q\|_2 \rho_r \;=:\; U_r(q). \tag{6}$$

Proof. Write $q^\top k_j = q^\top c_r + q^\top (k_j - c_r)$. By Cauchy–Schwarz, $|q^\top (k_j - c_r)| \leq \|q\|_2 \, \|k_j - c_r\|_2 \leq \|q\|_2 \rho_r$ using (4). Therefore $q^\top k_j \leq q^\top c_r + \|q\|_2 \rho_r = U_r(q)$. □

Equation (6) is a *certified* bound: never violated by construction. §5.3 reports the empirical violation count (zero) and the slack statistic $\mathrm{slack}_r(q) = U_r(q) - \max_{j \in B_r} q^\top k_j \geq 0$.

3.4 Selection policies

We compare two selection functions, both returning a k-subset of blocks.

FixedTopK.

$$S_{\text{fix}}(q) \;=\; \arg\operatorname*{top\text{-}k}_{r}\big(q^\top c_r\big). \tag{7}$$

Selection depends only on coarse summary scores. This is the natural baseline selector and is closely related to per-page criticality scoring in prior work.

BoundScreen.

$$S_{\mathrm{bs}}(q;\delta) \;=\; \arg\operatorname*{top-k}_{r}\big(q^{\top}c_r + \delta\cdot\|q\|_2\rho_r\big), \qquad \delta\in[0,1]. \tag{8}$$

At $\delta = 0$ this reduces to FixedTopK. At $\delta = 1$ selection ranks blocks by the certified upper bound U_r on their best possible token-level score. We use $\delta = 1$ throughout unless otherwise noted. The distinction is conceptually small but load-bearing: FixedTopK ranks blocks by what is *typical* inside them (the coarse dot product); BoundScreen ranks blocks by what is *possible* inside them (a Cauchy–Schwarz upper bound on the best token-level score).

3.5 Systems implementation

Three design choices carry most of the performance story.

Shared-across-queries selection (gather_shared). Within a single forward pass, all queries in a head share one selection S, scored from the head's mean query $\bar{q} = \frac{1}{L_q}\sum_{i=1}^{L_q} q_i$. The gathered key/value tensors then have shape (B, H_q, kb, d) — no L_q axis — which lets the post-gather attention use a single fused `scaled_dot_product_attention` call on a drastically reduced KV set. Per-query causal masking is applied inside that single call, so individual queries never attend to gathered tokens in their own future. The alternative — per-query selection and gather — is bit-equivalent to a masked reference implementation but runs approximately 12× slower on the ROCm hardware we used, because `torch.gather` with scattered reads along a large L_q axis does not coalesce. We discuss the retrieval consequences of this choice in §6.

Residual-refine packing. Unselected-block summaries are concatenated onto the selected key/value tensors as $m - k$ additional virtual tokens, giving a joint softmax of size $kb + m - k$. This retains (3) without requiring a separate forward pass, and keeps every query in causal contact with every past block — either via exact tokens from the selected blocks or via a one-vector summary from each unselected one.

Radius caching. In the decode regime, each generation step extends the KV cache by one token. All blocks' centres and radii except the last are unchanged; the cache recomputes only the growing block:

$$(c_r^{(t+1)}, \rho_r^{(t+1)}) = \begin{cases} (c_r^{(t)}, \rho_r^{(t)}) & r < r^\star, \\ (\text{recomputed from } K_r^{(t+1)}, V_r^{(t+1)}) & r = r^\star, \end{cases} \qquad r^\star = \left\lfloor \tfrac{n^{(t+1)}-1}{b} \right\rfloor. \tag{9}$$

This removes $O(nd)$ redundant work per decode step for the bound selector, which is what lets BoundScreen's decode wall-clock catch up to FixedTopK's despite carrying the extra radius computation.

3.6 Complexity

Per query head, let n be the KV length, b the block size, $m = \lceil n/b \rceil$ the block count, k the exact budget, d the head dimension.

$$\text{per-query cost:} \qquad O\Bigg(\underbrace{md}_{\text{scoring}} + \underbrace{kbd}_{\text{exact-refine}}\Bigg) = O\Big(\Big(\frac{n}{b} + kb\Big)\, d\Big), \tag{10}$$

$$\text{prefill cost } (n \text{ queries}): \qquad O\Big(\frac{n^2 d}{b} + nkbd\Big). \tag{11}$$

Prefill is *reduced-quadratic*, not linear: the n^2/b prefix persists because each query still scores every past block. The ratio of adaptive prefill to dense prefill $O(n^2 d)$ is

$$R(n, b, k) \;=\; \frac{n^2 d/b + nkbd}{n^2 d} \;=\; \frac{1}{b} + \frac{kb}{n}. \tag{12}$$

At $b = 64$, $k = 6$, $n = 32$k, $R \approx 0.028$ — an *algorithmic ceiling*, not a speedup claim. It is what the method could achieve if coarse scoring, gather, causal-mask construction, and residual summary packing were free. The measured 5.30× speedup at 32k in §5.1 is what remains after those costs; the gap between ceiling and measurement is concrete implementation surface, not a theoretical loss.

Minimising R over b gives $b^\star = \sqrt{n/k}$ with $R^\star(n, k) = 2\sqrt{k/n}$. For Qwen3-0.6B at $n = 32$k, $k = 6$, this yields $b^\star \approx 73$; the constraint $b \in \{8, 16, 32, 64\}$ used elsewhere in the paper makes $b = 64$ the nearest admissible value, which is what we use.

4 Experimental Setup

Model and runtime. All experiments use Qwen3-0.6B [Qwen Team, 2025] (28 decoder layers, $d_{\text{model}} = 1024$, 16 attention heads and 8 KV heads with grouped-query attention, bf16). We do not fine-tune the model; adaptive attention is applied as a drop-in replacement of the attention forward on selected layers. The primary hardware is a single AMD Ryzen AI Max+ 395 (Strix Halo, gfx1151) with 128 GB unified memory, running PyTorch 2.10 with ROCm 7.2 and the experimental AOTriton SDPA kernels enabled (`TORCH_ROCM_AOTRITON_ENABLE_EXPERIMENTAL=1`). Disabling that flag places the baseline on a slow math fallback and inflates every reported speedup by $\approx 3\times$; we report only numbers from the fast-path configuration.

Methods compared. Three families throughout: *dense baseline* (stock SDPA [Dao et al., 2022]), *FixedTopK* (Eq. 7), and *BoundScreen* (Eq. 8), all at $b = 64$ in residual-refine mode. Exact-budget sweeps use $k \in \{4, 5, 6, 8\}$; long-context latency sweeps focus on $k \in \{6, 8\}$.

Metrics. Prefill latency is reported as the median of ≥ 3 timed iterations (10 at context lengths up to 8k; 5 at 16k; 3 at 32k and 64k) following ≥ 3 warm-up iterations in the same process. Error bars ($\pm$) denote standard deviation within a single session. Calibration perplexity is measured on a fixed information-theory passage over non-overlapping 512-token windows:

$$\text{PPL} \;=\; \exp\left(-\frac{1}{N-1} \sum_{t=1}^{N-1} \log p_\theta(x_{t+1} \mid x_{1:t}) \right). \tag{13}$$

Passkey retrieval follows the standard needle-in-haystack protocol [Liu et al., 2024] with a random 5-digit code injected at depth $d \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ into a filler passage, scored by teacher-forced greedy correctness over five seeds per cell.

Artifact protocol. Every table and figure in this paper is regenerated by a single script from the JSON artifacts in `results/`. Where the same (method, context length) cell appears in multiple bench files, we keep the sample with the tightest within-session standard deviation. Source code, configs, and raw result files are released with the paper.

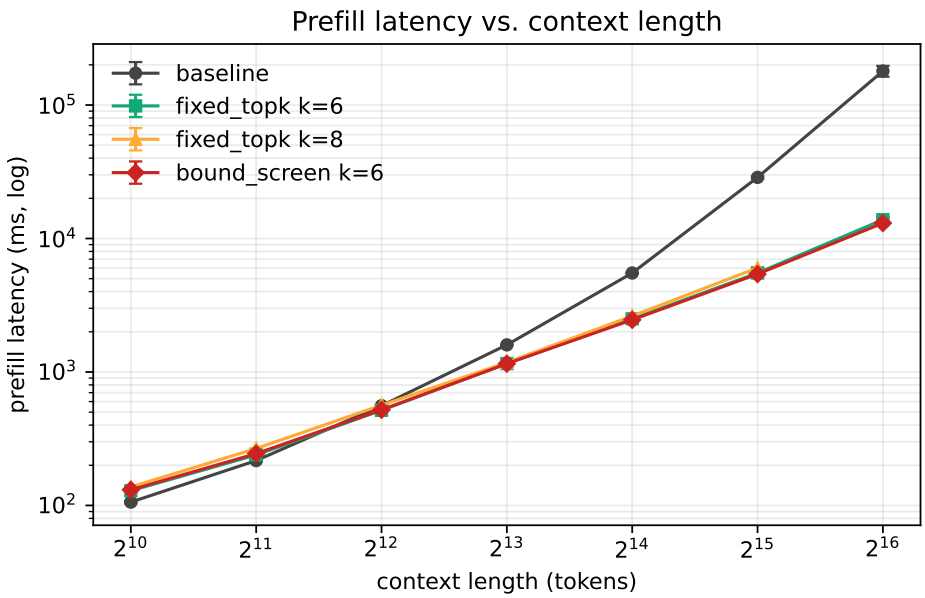

Figure 2: Prefill latency vs context length (Fig. 2). Log–log. Error bars are within-session standard deviations. BoundScreen at $k = 6$ crosses baseline between 2k and 4k and the gap widens at longer context. The 65k point exceeds Qwen3-0.6B's training context and appears in the appendix only.

ctx	baseline	FixedTopK k=6	FixedTopK k=8	BoundScreen k=6	speedup
1,024	105.9 ± 2.3	128.8 ± 0.5	137.6 ± 0.8	131.0 ± 1.1	0.81×
2,048	216.9 ± 3.8	239.1 ± 1.0	267.0 ± 0.8	244.6 ± 1.2	0.89×
4,096	558.8 ± 4.3	516.3 ± 1.0	564.0 ± 2.0	520.3 ± 1.2	**1.07**×
8,192	1596.0 ± 11.2	1155.6 ± 1.6	1178.6 ± 2.1	1154.1 ± 3.2	**1.38**×
16,384	5514.7 ± 33.4	2501.6 ± 13.1	2628.8 ± 7.8	2458.4 ± 9.1	**2.24**×
32,768	28703.0 ± 563.5	5517.6 ± 10.4	6027.4 ± 11.6	5415.4 ± 14.6	**5.30**×

Table 1: Prefill latency vs context length (Table 1). Milliseconds; median ± within-session standard deviation. Bold rows mark BoundScreen outperforming the dense baseline. The 65,536 row appears in the appendix because it exceeds Qwen3's training context.

5 Main Results

5.1 Long-context latency–quality tradeoff

Define the empirical speedup at context length n as

$$\text{Speedup}(n) \;=\; \frac{T_{\text{baseline}}(n)}{T_{\text{BoundScreen}}(n)}. \tag{14}$$

Results are given in Table 1 and Figure 2. The headline row is BoundScreen at $k = 6$:

$$\text{Speedup(4k)} = 1.07\times, \;\; \text{Speedup(8k)} = 1.38\times, \;\; \text{Speedup(16k)} = 2.24\times, \;\; \text{Speedup(32k)} = 5.30\times.$$

Crossover against the dense baseline occurs between 2k and 4k: below it, the coarse-score computation and mask bookkeeping dominate; above it, the $O(n^2/b)$ scaling of coarse scoring pulls ahead of dense $O(n^2)$ attention, and the gap widens at the rate predicted by (12). At 32k, a 28.7 s dense prefill collapses to 5.4 s.

Calibration perplexity is preserved. The Qwen3-0.6B baseline PPL is 11.096; BoundScreen at $k = 6$ measures 11.105, and FixedTopK at $k = 8$ measures 11.113. These differences sit

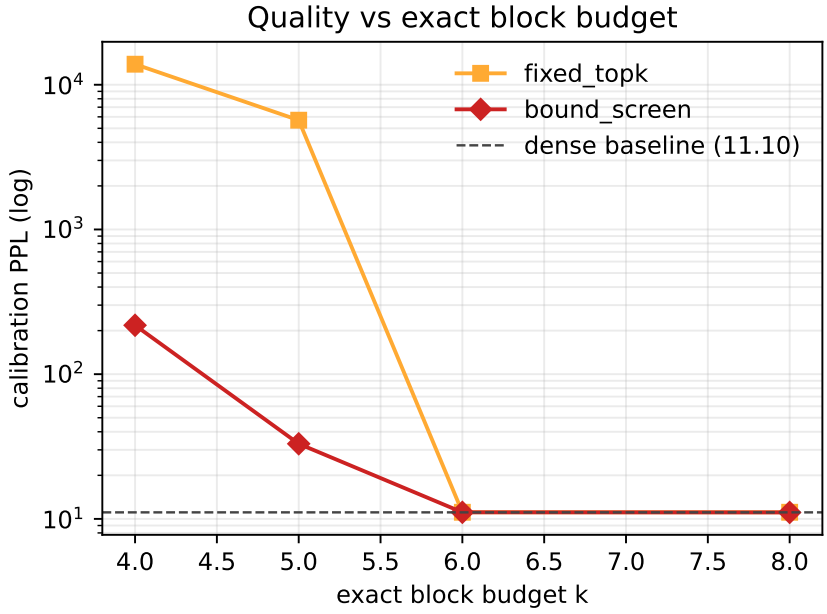

Figure 3: Calibration perplexity vs exact budget (Fig. 3). Log-y. Both selectors meet baseline at $k \geq 6$. Below that, FixedTopK degrades catastrophically; BoundScreen degrades, but by roughly two orders of magnitude less at $k = 4$.

k	FixedTopK PPL	BoundScreen PPL
4	13,865	217.8
5	5,689	33.042
6	11.113	11.105
8	11.113	11.086

Table 2: PPL vs exact block budget (Table 2). Block size 64, residual-refine. Baseline PPL = 11.096. Contribution: graceful degradation at $k \leq 5$, not lower budget at iso-quality.

well within run-to-run jitter on a single-passage calibration metric; we describe them as "no measurable loss," not as an improvement.

Intra-session variance is small: standard deviations are under 15 ms at every length up to 32k (about 0.5 % of the median at 32k). The one exception is the 64k baseline, whose large standard deviation reflects the position-embedding regime rather than the measurement protocol; we omit that row from quality-bearing claims.

5.2 Graceful degradation under tight exact budgets

Table 2 and Figure 3 summarise the quality-vs-budget behaviour at $b = 64$. Both selectors meet baseline calibration perplexity at $k \geq 6$; the contribution is below that operating point.

At $k = 5$, FixedTopK loses three orders of magnitude of perplexity ($11 \to 5{,}689$); BoundScreen loses less than one ($11 \to 33$). At $k = 4$, FixedTopK reaches $\approx 10^4$ while BoundScreen is two orders of magnitude lower. Quantitatively, the log-PPL gap between selectors at matched budget is

$$\log_{10} \frac{\mathrm{PPL}_{\mathrm{FixedTopK}}}{\mathrm{PPL}_{\mathrm{BoundScreen}}} \approx \begin{cases} 1.80 & k = 4, \\ 2.24 & k = 5. \end{cases} \tag{15}$$

Neither selector is usable at $k \leq 5$ in the strict sense — both fall below any reasonable quality floor — but the *failure mode* is qualitatively different: FixedTopK drops off a cliff, BoundScreen

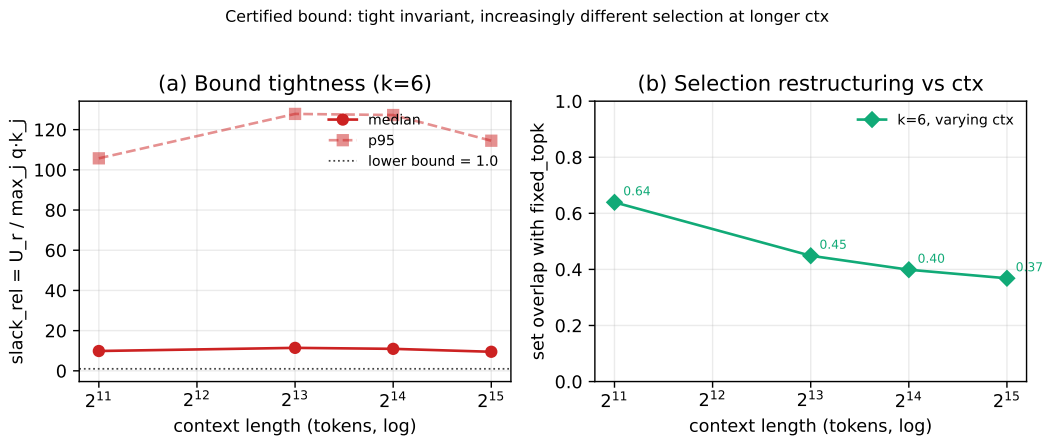

Figure 4: Bound tightness and selection restructuring (Fig. 4). (a) Median and $p95$ of $U_r(q)/\max_j q^\top k_j$ across probed queries; always ≥ 1 by Prop. 1, and stable around 10 across context lengths. (b) Set overlap between BoundScreen and FixedTopK selected blocks at $k = 6$; divergence grows with context length, reaching 0.37 at 32k.

degrades.

We frame the contribution as a change of failure mode, not as lower-budget behaviour at matched quality. At the operating point where an exact budget is provisioned conservatively enough to be correct on its headline workload, both selectors work. Under provisioning error — a shorter budget than expected, a workload with less average-case redundancy than expected — BoundScreen continues to produce coherent output at perplexities that are two orders of magnitude better than the coarse-ranking alternative. This is the behaviour the bound is designed to deliver: a block whose best possible token-level score is large is never silently dropped simply because its average score is unremarkable.

5.3 Bound behaviour and selector restructuring

Definition 2 (bound slack)**.** For a query q and block r, define

$$\text{slack}_r(q) \;=\; U_r(q) - \max_{j \in B_r} q^\top k_j \;\geq\; 0, \qquad \text{slack-rel}_r(q) \;=\; \frac{U_r(q)}{\max_{j \in B_r} q^\top k_j} \;\geq\; 1. \tag{16}$$

Definition 3 (selection overlap)**.** For selectors S_1, S_2 at the same budget k,

$$\text{overlap}_k(q) \;=\; \frac{|S_1(q) \cap S_2(q)|}{k} \;\in\; [0, 1]. \tag{17}$$

We report the mean over probed queries and blocks.

Violations and slack. For every (q, r) pair probed, Proposition 1 requires $\text{slack-rel}_r(q) \geq 1$. Across every probed configuration — context lengths 2k, 8k, 16k, 32k at $k = 6$, plus $k \in \{4, 8\}$ at 2k — the ratio is ≥ 1 on every sampled cell: zero observed violations (Figure 4a). The median ratio stays in the 9.5–10.9 range across all contexts, indicating that the bound is loose in absolute terms (roughly an order of magnitude above the actual block-max logit) but remarkably stable. Slack does not widen with context length, consistent with the derivation of (6) having no n-dependence.

Restructuring vs FixedTopK. Figure 4b plots overlap_6 between BoundScreen and FixedTopK as context grows: 0.64 at 2k, 0.45 at 8k, 0.40 at 16k, 0.37 at 32k. By 32k, 63 % of BoundScreen's six selected blocks are not in FixedTopK's top-6. This rules out the interpretation

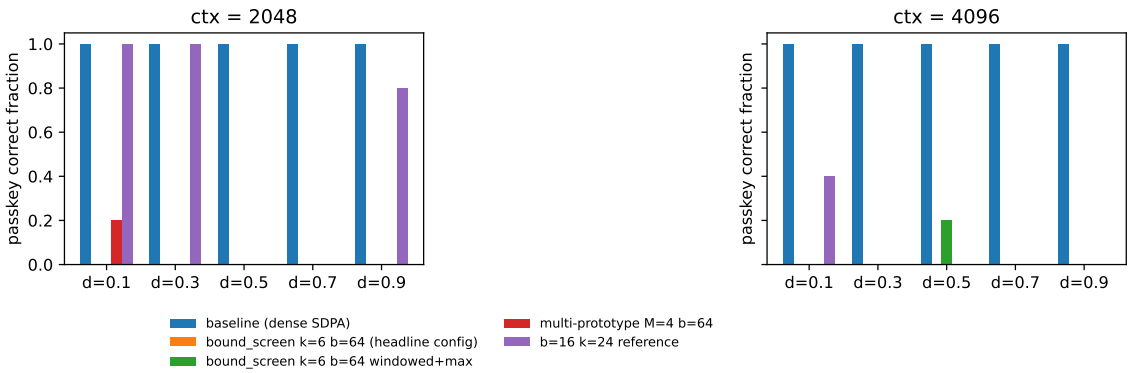

Figure 5: Passkey retrieval correctness by depth (Fig. 5), at 2k and 4k context. Baseline is dense SDPA. All $b = 64$ adaptive variants are at or near zero across all depths. The $b = 16$ reference recovers retrieval at easy depths only; the mechanism is analysed in §6.2.

that BoundScreen is a relabelling of FixedTopK: the two selectors make measurably different picks at long context. §5.2 is a separate experiment — different k, different context — but shows that the selectors' behaviours also differ under tight exact budgets, in the direction the bound would predict. We do not directly attribute the graceful degradation to the specific picks the overlap measurement identifies; we state only that (i) the two selectors are empirically distinct and (ii) BoundScreen is the one that preserves graceful degradation.

6 Limitation Analysis

6.1 Passkey retrieval failure at $b = 64$

We probe sparse single-token retrieval with a standard passkey test [Liu et al., 2024]: a 5-digit magic number is injected at depth $d \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ into a filler passage of context length $n \in \{2048, 4096\}$, and a trailing query "what is the magic number? A:" elicits a teacher-forced completion. Correctness is the fraction of trials (over five seeds) whose greedy completion matches the injected digits token-by-token. Results appear in Figure 5.

The dense SDPA baseline scores 1.00 at every (context, depth) cell. Every adaptive variant at $b = 64$ scores at or near zero. BoundScreen at $k = 6$ with shared-mean selection: zero across every cell. BoundScreen with windowed-max aggregation (a per-query-window variant): zero except a single 0.20 cell at $4\text{k}/d = 0.5$. BoundScreen with $\delta = 5$ (radius-term-dominant selection): zero. A reference configuration at $b = 16, k = 24$ — using the same gather budget in tokens ($kb = 384$) but a finer block partition — recovers 1.00 correctness at $d = 0.1$ and 0.80 at $d = 0.9$, but still fails at mid-depth.

We state this plainly: the method as built is not a general retrieval architecture. The pattern of failure is not a selector tuning artefact.

6.2 Mechanism

The failure has two mechanisms that compound.

Summary dilution. Consider a block whose keys are one informative *needle* key $k^\star$ plus $b - 1$ filler keys $\{k_j\}$, and write $\mathbb{E}[k_j] = \mu$, $\mathrm{Var}[k_j] = \sigma^2$ for the filler distribution. The mean summary is

$$c_r \;=\; \frac{1}{b}\left(k^\star + \sum_{j \neq j^\star} k_j\right) \;\approx\; \mu + \frac{k^\star - \mu}{b}, \tag{18}$$

M	passkey cells > 0	best cell	PPL	8k prefill
1	0 / 10	0 %	11.082	2,210 ms
2	0 / 10	0 %	11.114	3,374 ms
4	1 / 10	20 %	11.084	4,624 ms
8	0 / 10	0 %	—	—
$b = 16, k = 24, M = 1$ (ref.)	6 / 10	100 %	11.110	4,572 ms

Table 3: Multi-prototype at $b = 64, k = 6$ (Table 3), residual-refine, windowed max, compared to a fine-block reference. Retrieval does not improve; latency regresses monotonically with M. Calibration perplexity is preserved throughout.

so the needle contributes $O(1/b)$ to the center direction. For a query q whose retrieval signal lies in the $k^\star - \mu$ direction, the coarse-score gap between the needle block and a pure-filler block is $O(1/b)$. At $b = 64$ this is small enough that the query's coarse-score ordering is dominated by filler-direction noise rather than by the needle contribution. The radius term $\|q\|_2\rho_r$ in (6) does grow when the block contains an outlier — but for retrieval queries at this model scale, $q^\top c_r$ dominates the ranking. Raising δ to 5 (radius-term-dominant selection) does not rescue this: the invariant (4) still holds, but the radius term alone is not discriminative enough to distinguish a needle block from its neighbours.

Selection granularity. A 64-token block is the unit of selection. Keeping it requires outscoring 31 other blocks at 2k with six picks. At $b = 16$ the same argument operates against 127 blocks with 24 picks — a four-times-larger selection budget at four-times-finer granularity — and a 1/16 dilution is more often distinguishable from a 0/16 filler block than 1/64 is from 0/64. The $b = 16$ reference in §6.1 recovers retrieval at easy depths under exactly this accounting.

Neither mechanism alone is decisive; together they are.

6.3 Negative result: multi-prototype

A natural fix is to split each block into M sub-blocks, store one $(c_{r,m}, \rho_{r,m})$ pair per sub-block, and bound the block via

$$U_r^{(M)}(q) \;=\; \max_{m=1,\dots,M} \big(q^\top c_{r,m} + \|q\|_2 \rho_{r,m}\big). \tag{19}$$

This remains a certified upper bound: the pointwise max of upper bounds is an upper bound on the pointwise max. Formally, for $j \in B_{r,m}$,

$$q^\top k_j \overset{\text{Prop. 1}}{\leq} q^\top c_{r,m} + \|q\|_2 \rho_{r,m} \;\leq\; \max_{m'} \big(q^\top c_{r,m'} + \|q\|_2 \rho_{r,m'}\big) = U_r^{(M)}(q),$$

so $\max_{j \in B_r} q^\top k_j \leq U_r^{(M)}(q)$. We implemented (19) for $M \in \{2, 4, 8\}$ with equal sub-block splits and verified the invariant holds in every probe.

Table 3 shows that retrieval is not restored and that latency regresses monotonically with M: the M-fold increase in scoring cost is not amortised by any reduction in the downstream attention shape, which still sees $kb + m$ keys. Calibration PPL is preserved, consistent with multi-prototype leaving average-case modelling behaviour intact.

The precise lesson: summary resolution alone is insufficient when selection granularity stays coarse. Multi-prototype gives each block M (center, radius) pairs, so per-prototype summaries cover b/M tokens — as distinctive as a finer block would produce. But the selector still picks k blocks out of m, not kM prototypes out of mM. The passkey needs finer selection as well as finer summaries. Any fix that recovers selection resolution by treating each prototype

as a selectable unit collapses to "use block size b/M" and pays the $\lceil n/(b/M) \rceil$ summary count in the softmax key set — which is the latency cost we saw in the $b = 16$ reference row.

6.4 Interpretation

The method is effective for long-context inference under calibration/PPL-style objectives: the selector preserves average modelling behaviour, the bound gives real graceful degradation under under-provisioned exact budgets, and the systems implementation delivers growing wall-clock wins at longer contexts. It is *not* a mechanism for retrieving sparse single-token information at the coarse block granularities tested here.

The correct positioning is the framing sentence: this is a certified runtime screening method for long-context efficiency, not a general retrieval architecture. Quest [Tang et al., 2024] and our method use different per-block metadata with different tradeoffs: Quest maintains per-page element-wise key min/max ($2d$ scalars per block) and scores against those min/max bounds — finer-grained per-token information at the cost of more scoring operations; we store one center and one scalar radius ($d + 1$ scalars) and derive our bound from Cauchy–Schwarz — coarser metadata and a cheaper scoring pass, but a provable block-level upper bound in one line. Neither design point dominates the other; architectures such as DuoAttention [Xiao et al., 2024], which separate retrieval heads from streaming heads, suggest that a production system wanting both sparse-retrieval fidelity and the present method's long-context wall-clock behaviour would likely compose them.

7 Discussion

Where the method is useful. Long-context prefill of existing dense pretrained models at context lengths of $\sim$ 4k and above, where the $O(n^2/b)$ scaling of coarse scoring comfortably beats dense $O(n^2)$ attention; runtime-efficient approximate exact attention for latency-sensitive long-context inference; scenarios where average-modelling behaviour and graceful degradation under tight budgets matter more than pinpoint single-token retrieval.

Where it is not the right tool. Tasks that require recovering a specific distant token with high probability — passkey retrieval [Liu et al., 2024], exact code-span lookup, fact extraction from long documents — at least at the 64-token block granularity used here. Applications of this kind warrant finer-grained per-page metadata along the lines Quest [Tang et al., 2024] uses, or a dedicated retrieval mechanism orthogonal to the present work.

Future directions. Finer selection granularity without the residual summary cost (e.g. hierarchical selection, or decoupling summary granularity from gather granularity); tighter bounds than the isotropic Cauchy–Schwarz form (axis-aligned box bounds along the lines of Quest's per-dimension metadata, Mahalanobis-style whitening); hybrid stacks that apply certified screening only to layers where it helps and dense attention elsewhere, analogous to head-level routing in DuoAttention [Xiao et al., 2024]. We explicitly do not speculate on fine-tuning the selector; every result in this paper is from a single frozen dense model.

8 Conclusion

We showed that certified block screening enables a practical adaptive long-context attention mechanism with strong wall-clock gains at context lengths of 4k and above, reaching a 5.30× prefill speedup over a dense SDPA [Dao et al., 2022] baseline at 32k tokens on Qwen3-0.6B [Qwen Team, 2025] while preserving calibration perplexity and improving under-budget robustness. We also identified a clear limitation: coarse block screening at 64-token granularity does not recover

sparse single-token retrieval, and simple multi-prototype extensions fail to rescue retrieval because selection granularity, not summary granularity, is the limiting factor. These results position certified block screening as a useful runtime efficiency tool rather than a universal replacement for exact long-range retrieval.

References

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Qichen Fu, Minsik Cho, Thomas Merth, Sachin Mehta, Mohammad Rastegari, and Mahyar Najibi. LazyLLM: Dynamic token pruning for efficient long context LLM inference. *arXiv preprint arXiv:2407.14057*, 2024.

Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Hayden Kwok-Hay So, Ting Cao, Fan Yang, and Mao Yang. SeerAttention: Learning intrinsic sparse attention in your LLMs. *arXiv preprint arXiv:2410.13276*, 2024.

Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 2024.

Qwen Team. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.